

1. **The document:** What HTML is for · The document skeleton · What goes in the head
2. **Elements and attributes:** Elements, tags and comments · Attributes · Nesting and the document tree
3. **Headings, text and lists:** Headings and the outline · Paragraphs and text-level elements · Lists
4. **Semantic HTML:** Why semantic HTML · The landmark elements
5. **Block, inline, div, span:** Block and inline elements · div and span: generic containers
6. **Links and images:** Links and paths · Images and alt text
7. **Tables:** Tables for tabular data
8. **Forms:** Forms: labels, inputs, names and buttons · Built-in form validation
9. **Validate and inspect:** Validate and inspect

The document

What HTML is for

HTML (HyperText Markup Language) describes the structure and meaning of a page: it marks each piece of content as a heading, a paragraph, a list, a link, a form and so on.

Example. The words "Opening hours" can be a heading (`<h2>Opening hours</h2>`), a paragraph or a list item. Only as a heading does a screen reader announce "heading level 2, Opening hours" and let the user jump to it with one key. Same words, different meaning, and the markup is what carries it.

Common mistake. Choosing tags by how they look: `<h3>` because "the text should be smaller", `<blockquote>` to indent a paragraph. The look is CSS's job and can be changed in one line; the tag must say what the content is.

The document skeleton

Every HTML page starts with `<!DOCTYPE html>` and has one `<html>` element that contains a `<head>` (information about the page) and a `<body>` (the content shown in the window).

HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Home · Riverside FC</title>
  </head>
  <body>
    <h1>Riverside FC</h1>
    <p>Welcome to the official club website.</p>
  </body>
</html>
```

Example. Open the skeleton above in a browser: the tab shows "Home · Riverside FC" (from `<title>`, in the head), and the window shows only the heading and the paragraph (from the body). The indentation is for people; the browser ignores it.

Common mistake. Leaving out `<!DOCTYPE html>` because "the page looks fine without it". The difference shows up later: in quirks mode some CSS sizing rules behave like in very old browsers, and the validator reports an error. Start every file with it.

What goes in the head

The `<head>` holds metadata, data about the page: the character encoding, the viewport settings, the title, a description, and links to stylesheets and scripts; none of it is displayed as page content.

- `<meta charset="utf-8">`: the file is encoded in **UTF-8**, which covers every alphabet, accents and emoji. Without it, "España" can appear as "Españ±a".

- `<meta name="viewport" content="width=device-width, initial-scale=1">`: on phones, lay the page out at the real screen width. Without it mobile browsers pretend the screen is about 980 px wide and shrink everything, so responsive CSS seems not to work.
- `<title>`: the text of the browser tab, of bookmarks and of the link in search results; screen readers read it first. It is **required**: the validator reports an error if it is missing.
- `<meta name="description" content="...">`: a one-sentence summary that search engines may show under the title.
- `<link rel="stylesheet" href="styles.css">`: connects an external CSS file (see Writing CSS).
- `<script src="app.js" defer></script>`: loads a JavaScript file; `defer` runs it after the HTML has been parsed, so the script can find every element (see Loading scripts).

Example. The fixtures page of a club site: `<title>Fixtures · Riverside FC</title>` in the head names the tab, while `<h1>Fixtures and results</h1>` in the body is the visible heading. A good title puts the specific part first, so it still reads well when many tabs are open and truncated.

Common mistake. Mixing up three different "titles": the `<title>` element (in the head, names the document in the tab), the `<h1>` element (in the body, the visible main heading) and the `title` attribute (a tooltip on one element). They have different places and different jobs.

Elements and attributes

Elements, tags and comments

An **element** is a start tag, its content and an end tag (`<p>Hello</p>`); the **tags** are only the markers in angle brackets. **Void elements** such as `<br>`, `<img>` and `<input>` have no content and no end tag.

Example. In `<a href="squad.html">See the squad</a>` the **start tag** is `<a href="squad.html">` (it carries an attribute), the **content** is "See the squad", the **end tag** is `</a>`, and all three together are the a **element**.

Common mistake. Relying on the browser to close your tags. Browsers repair missing end tags silently, so the page often looks right, but the repaired tree may not be what you meant (text ending up inside the wrong element), the validator reports errors, and CSS and JavaScript later target the wrong elements.

Attributes

Attributes add information or settings to an element; they are written as `name="value"` pairs inside the start tag, separated by spaces.

Attributes you will use on your first pages

Attribute	On	Purpose
<code>href</code>	<code><a></code> , <code><link></code>	Where the link points
<code>src</code>	<code></code> , <code><script></code>	Which file to load
<code>alt</code>	<code></code>	Text alternative for the image
<code>width / height</code>	<code></code>	Display size in pixels, reserves space
<code>lang</code>	<code><html></code> and any element	Language of the content
<code>id / class</code>	Any element	Unique name / reusable group name
<code>title</code>	Any element	Advisory tooltip text
<code>for</code>	<code><label></code>	The id of the control it labels
<code>type, name</code>	<code><input></code> , <code><button></code>	Kind of control; key of the submitted value

Example. `` has four attributes: the file to load, the text alternative, and the size the browser reserves for it. Their order does not matter.

Common mistake. Giving the same `id` to several elements (`id="card"` on every card). An `id` must be unique: a fragment link, a `<label for>` or JavaScript expects exactly one match. For a group of similar elements use a `class`: `class="card"`.

Nesting and the document tree

Elements nest inside each other and form a tree; each element must be **closed inside the element that contains it** (never overlapping), and some elements accept only certain children.

Example. In `<body><header><h1>Blog</h1><nav>...</nav></header><main>...</main></body>`, `<body>` is the parent of `<header>` and `<main>`, which are siblings; `<h1>` and `<nav>` are children of `<header>` and descendants of `<body>`.

Common mistake. Putting a list or a heading inside a `<p>`. The browser ends the paragraph as soon as it meets the `<ul>`, so the list is not inside the paragraph, and the leftover `</p>` creates an extra empty paragraph. The Try-it box shows three bordered paragraphs where the code seems to have two.

Headings, text and lists

Headings and the outline

`<h1>` to `<h6>` mark headings of six levels; together they form the **outline** (the table of contents) of the page, so the level must follow the structure, not the font size you want.

Example. The outline in the Try-it box reads like a table of contents: page title, two chapters, two sections in each. If "New signing announced" were an `<h4>`, the outline would have a level-4 heading directly under a level-2 one, and a screen-reader user would wonder what was missing.

Common mistake. Picking `<h4>` for a subtitle because "it has the right size", or using a bold paragraph as a fake heading. The first breaks the outline; the second is invisible to heading navigation. Choose the level by structure and adjust the size with CSS.

Paragraphs and text-level elements

Running text goes in **paragraphs** (`<p>`); inside a line, text-level elements add meaning: `<strong>` (importance), `<em>` (stress emphasis), `<a>` (link), `<br>` (a line break that belongs to the content).

Example. "I did **not** say that" and "I did not say **that**" mean different things when spoken; `<em>` marks where the stress goes: `<p>I did not say that.</p>`. A warning that must not be missed is `<strong>`.

Common mistake. Using `<br><br><br>` or empty `<p></p>` to create vertical space. They add meaningless empty lines to the content, and the spacing cannot adapt to the screen. Separate the content into proper elements and add space with CSS margin (see [the box model](#)).

Lists

`<ul>` makes an **unordered (bulleted) list** and `<ol>` an **ordered (numbered) list**; each item is an `<li>`. A nested list goes **inside the** `<li>` it belongs to.

Example. A recipe page: the ingredients are a `<ul>` (the order you buy them in does not matter), the method is an `<ol>` (step 3 cannot come before step 1), and the site menu at the top is a `<ul>` of links inside a `<nav>`.

Common mistake. Placing a sub-list next to the items instead of inside one: `<ul><li>Goalkeepers</li><ul>...</ul></ul>`. A list may only contain `<li>` children, so the validator reports an error; move the inner `<ul>` inside the `<li>`, before its `</li>`.

Semantic HTML

Why semantic HTML

Semantic HTML means choosing elements whose name says what the content is (`<nav>`, `<article>`, `<button>`) instead of generic boxes (`<div>`, `<span>`); it makes pages accessible, understandable to search engines and easier to maintain.

- **Accessibility:** screen readers list landmarks ("navigation", "main") and headings, so users jump straight to what they need. A real `<button>` works with the keyboard for free; a `<div>` made clickable does not.

- **SEO** (search engine optimisation): search engines read the title, headings and main content to understand what the page is about. Clear structure helps them; it is not a magic ranking boost.
- **Maintainability**: `</article>` tells a teammate what is closing; `</div></div></div>` does not.

HTML

```
<!-- "div soup": works, but means nothing -->
<div class="top">
  <div class="title">Riverside FC</div>
  <div class="menu">...</div>
</div>

<!-- semantic: the same page, with meaning -->
<header>
  <h1>Riverside FC</h1>
  <nav>...</nav>
</header>
```

Example. A screen-reader user opens both versions above. On the semantic one they hear "banner landmark, heading level 1, Riverside FC, navigation landmark" and can press a key to jump to the navigation. On the `<div>` version they hear only "Riverside FC" followed by the menu text, with no way to know it is a title or a menu.

Common mistake. Using semantic elements as decoration: wrapping every block in `<section>` or every image in `<article>`. Each element is a claim about the content, and a wrong claim misleads as much as none. When no element fits, a `<div>` is the honest choice.

The landmark elements

Landmark elements divide a page into regions: `<header>` (introduction, logo, site navigation), `<nav>` (major navigation links), `<main>` (the unique main content), `<section>` (a themed group with a heading), `<article>` (a self-contained piece), `<aside>` (related but secondary content) and `<footer>` (closing information).

Which landmark?

Element	Question to ask	Example
<code><header></code>	Is it the introduction of the page (or of an article)?	Logo, site name, main menu
<code><nav></code>	Is it a major set of navigation links?	Main menu, table of contents
<code><main></code>	Is it the content this page exists for?	Everything between header and footer
<code><section></code>	Is it a themed group I can give a heading to?	"Upcoming fixtures"
<code><article></code>	Would it make sense on its own elsewhere?	A match report, a product card, a comment
<code><aside></code>	Is it related but could be removed without loss?	League table widget, related links
<code><footer></code>	Is it closing information?	Copyright, contact, legal links

HTML

```
<body>
  <header>
    <h1>Riverside FC</h1>
    <nav><!-- main menu --></nav>
  </header>
  <main>
    <section>
      <h2>Latest news</h2>
      <article>
        <h3>Riverside 2-1 Northside</h3>
        <p>A late goal sealed the win...</p>
      </article>
    </section>
    <aside>
      <h2>League table</h2>
    </aside>
  </main>
  <footer>© 2026 Riverside FC</footer>
</body>
```

Example. In the code above, the match report is an `<article>` (it makes sense on its own, shared on social media), "Latest news" is a `<section>` (a group of reports with a heading), and the league table is an `<aside>` (useful, but the page still works without it). The tool below draws the outline of any page you paste.

Common mistake. Choosing between `<section>` and `<article>` by size or position. The question is independence: a match report stands alone, so it is an `<article>`; "Upcoming fixtures" is a themed group with a heading, so it is a `<section>`. An article can contain sections and a section can contain articles.

Block, inline, div, span

Block and inline elements

By default, **block elements** (`<p>`, `<h1>`, `<ul>`, `<div>`, the landmarks) **start on a new line and take the full available width**; **inline elements** (`<a>`, `<strong>`, `<em>`, `<span>`, `<img>`) **flow inside a line of text and take only the width of their content**.

Example. In the Try-it box, the solid navy outlines (blocks) stretch across the whole width even when the text is short; the dashed orange outlines (inline) hug their words. Add `display: block;` to the `span` rule and watch the two spans move onto separate lines.

Common mistake. Putting a block element inside an inline one, e.g. wrapping a whole card `<div>` in a `<span>`, or an `<h2>` in an `<em>`. Inline elements may contain only text-level content; the browser has to repair the tree and the validator reports an error. (`<a>` is the exception: it may wrap blocks.)

div and span: generic containers

`<div>` (block) and `<span>` (inline) are **generic containers with no meaning**: use them to group content for styling or scripting only when no semantic element fits.

Example. In the Try-it box each ticket is an `<article>` (it stands on its own), but the wrapper that places them side by side is a `<div>`: it exists only for layout and has no meaning. The prices are `<span>`s because a price is not more important than the text around it; the class is just a styling hook.

Common mistake. "Divitis": building the whole page from nested `<div>`s with class names such as header, nav and main. It looks identical, but screen readers and search engines see one big anonymous box. Use the landmark elements, and keep `<div>` for pure styling wrappers.

Links and images

Links and paths

The `<a>` element makes a link; its `href` is an **absolute URL** (another site), a **relative path** (a file of your own site, found from the current page) or a **fragment** `#id` (a place on a page).

- File names in paths are **case-sensitive** on most web servers, GitHub Pages included: `Squad.html` and `squad.html` are different files there, even though Windows treats them as the same. Use lowercase names without spaces.
- A fragment link needs a target: some element with a matching `id`.

Kinds of `href`, seen from `index.html` in the site folder

href	Kind	Goes to
<code>https://www.w3.org/</code>	Absolute URL	Another site; the scheme <code>https://</code> is required
<code>squad.html</code>	Relative, same folder	The sibling file <code>squad.html</code>
<code>pages/tickets.html</code>	Relative, subfolder	Into the pages folder first
<code>../index.html</code>	Relative, parent folder	<code>..</code> means one folder up
<code>/index.html</code>	Root-relative	From the root of the server , not of your folder
<code>#results</code>	Fragment	The element with <code>id="results"</code> on this page
<code>fixtures.html#results</code>	File + fragment	Opens <code>fixtures.html</code> , scrolls to <code>id="results"</code>
<code>mailto:info@example.com</code>	Another scheme	Opens the email program

Example. A site with `index.html`, `fixtures.html` and a folder `pages/` containing `tickets.html`. From `index.html`: `href="pages/tickets.html"`. From `pages/tickets.html` back home: `href="../index.html"`. From `index.html` to the results section of the fixtures page: `href="fixtures.html#results"`.

Common mistake. Using root-relative paths (`/squad.html`) when opening the files straight from your disk. With `file://`, `/` means the root of the **drive** (e.g. `C:/`), not your project folder, so every link breaks; they only work once the site is served. While you work from local files, use plain relative paths such as `squad.html`.

Images and alt text

`<img>` **embeds an image**: `src` is the path of the file, `alt` a text alternative, and `width` and `height` the display size in pixels so the browser can reserve its space before the file arrives.

Example. For the club crest, `alt="Riverside FC crest: a blue shield with a white river"` gives the information. If the crest is also the link to the homepage, describe the destination instead: `alt="Riverside FC home"`. A decorative wave graphic between sections gets `alt=""`.

Common mistake. Writing `alt="image"`, `alt="photo of..."` or the file name. Screen readers already say "image"; the `alt` must carry the information the picture gives. For a chart, that may be its conclusion: "Season ticket sales doubled between 2023 and 2025".

Tables

Tables for tabular data

A **table** shows data organised in rows and columns: `<table>` contains rows (`<tr>`) of header cells (`<th>`) and data cells (`<td>`), with an optional `<caption>` and the groups `<thead>` and `<tbody>`.

Example. A fixtures list has a date, an opponent and a venue for each match: a table. A squad list with only names is a list (`<ul>`), not a one-column table. A "logo on the left, menu on the right" header is layout, which is CSS.

Common mistake. Making header cells with bold `<td>`s, or using a table to put things side by side. Bold data cells look like headers but are not headers for assistive technology; side-by-side placement is layout, done with CSS [Flexbox](#) and [Grid](#).

Forms

Forms: labels, inputs, names and buttons

A `<form>` groups controls whose values are sent together: each `<input>` needs a `<label>` (linked by `for` = the input's `id`) and a name (the key its value is sent under), and a `<button type="submit">` sends the form.

Example. Click the word "Password" in the Try-it box: the cursor jumps into the password field, because the label is linked to it. If this form used the default `get` method and was submitted, the browser would load `login.html?email=ana%40example.com&password=s3cret%21`: the password would be visible in the address bar and saved in the history. That is why a login form uses `method="post"`.

Common mistake. Believing a form without `action` "does nothing" when submitted. It **reloads the current page**, and with the default `get` method every named field, the password included, is appended to the URL. Before there is a server, give login forms `method="post"` and expect the page to reload.

Built-in form validation

HTML can check values before a form is sent: `required`, `minlength/maxlength`, `min/max`, `pattern` and typed inputs such as `type="email"` make the browser block the submission and show a message.

- `required`: the field cannot be empty (a boolean attribute).
- `minlength="8" / maxlength="64"`: limits on the number of characters of text.
- `min="1" / max="10"`: limits on numbers and dates.
- `pattern="[0-9]{5}"`: the value must match this regular expression (here, exactly five digits).

- `type="email"`, `type="url"`, `type="number"`: the value must have that format.

Example. In the Try-it box the empty required fields start red. Type `abc` as the password: it stays red because of `minlength="8"`; at the eighth character it turns green. The postcode is green while empty (it is not required) and red as soon as you type a letter.

Common mistake. Treating `required` and `type="email"` as protection. They improve the experience, but a user can remove them in DevTools in seconds and a tool like `curl` ignores them completely. Real validation always happens on the server.

Validate and inspect

Validate and inspect

The **W3C Markup Validator** checks your HTML against the rules of the standard and lists its errors; the browser **DevTools** (Elements tab) show the DOM the browser actually built from it.

Example. A page has `<p>Intro <ul><li>One</li></ul></p>`. The validator reports "No p element in scope but a p end tag seen". In the Elements tab you see why: the browser closed the paragraph before the list and turned the stray `</p>` into an extra empty `<p></p>`.

Common mistake. Taking the Elements tab as a copy of your file. It shows the **repaired, live** DOM: missing end tags appear "fixed", and a `<tbody>` appears in every table even if you never wrote one. To see what you really wrote, use View source or your editor, and run the validator.